

多智能体编排：模式、通信与失败边界

本文回答：什么时候才需要多智能体？有哪些编排模式？Agent 之间怎么通信？为什么多数生产系统不应该一上来就上多 Agent？

一、先问：真的需要多智能体吗

多智能体不是单 Agent 的“高级替代品”，而是用**通信成本、调试成本、Token 成本**换取**并行度与专业分工**。

在决定上多智能体前，先确认三件事：

1. 单 Agent 的失败，是因为“上下文太杂”，还是任务天然需要多个角色？
2. 任务是否可以被稳定拆成多个互不冲突的子任务？
3. 团队是否有能力观测和调试多 Agent 之间的消息流？

如果只是上下文杂乱，优先做 Context Engineering、Memory 分层或 Skill 化；如果任务可稳定拆解，优先用 Workflow / Graph，而不是多 Agent。

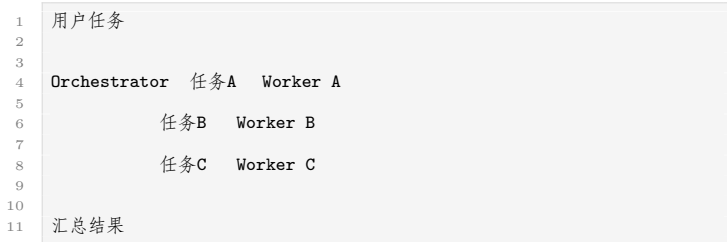
适合多智能体的信号：

- 子任务需要不同的 System Prompt、工具集或记忆边界；
- 子任务可并行，且互不阻塞；
- 需要一个 Agent 生成、另一个 Agent 审查的对抗式质量结构；
- 不同角色由不同团队维护，需要独立发布与权限隔离。

二、五种核心编排模式 + 一种特殊模式

1. Orchestrator-Workers (编排者-工作者)

结构：一个 Orchestrator 负责拆解任务、分发给多个 Worker，Worker 返回结构化结果，Orchestrator 汇总并交付。



- 优点：并行度高，子任务互不干扰，扩展容易。
- 缺点：Orchestrator 是单点，拆解质量决定全局上限。
- 适合：调研、报告生成、多源信息汇总、多模块代码生成。
- Worker 通常设计为无状态：输入任务卡，输出结果卡，不自行决定全局。

2. Supervisor (监督者)

结构：Supervisor 不拆全局计划，而是根据当前状态，在多个专业 Agent 之间路由，直到满足退出条件。

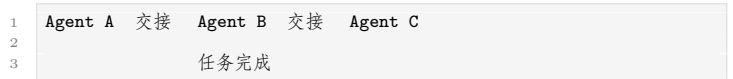


- 优点：动态路由，能根据中间结果换角色；比 Orchestrator 更灵活。

- 缺点：监督逻辑容易变成难以测试的 Prompt 黑洞。
- 适合：诊断、客服升级、复杂问题需要多轮角色切换的场景。
- 与 Orchestrator 的区别：Orchestrator 先拆后发，Supervisor 边看边路由。

3. Swarm / Handoff (动态交接)

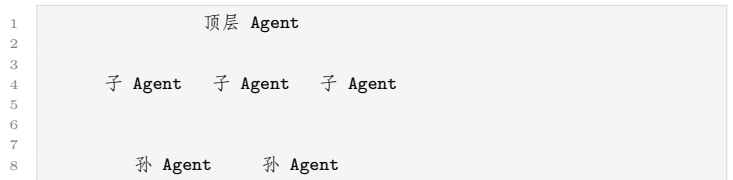
结构：没有中心调度者。Agent 之间按规则或模型判断进行交接，一次只有一个 Agent 活跃，直到任务完成。



- 优点：上下文随交接流转，交互自然，适合对话式客服。
- 缺点：交接判定难调，容易无限交接或上下文膨胀。
- 适合：客服转接、多领域助手、需要“把当前用户带到正确专家”的场景。
- 必须设计：最大交接次数、交接时携带的最小上下文、无路可走时的兜底 Agent。

4. Hierarchical (分层递归)

结构：高层 Agent 拆解任务后，子 Agent 可以继续拆解并生成下一级 Agent，形成树形递归结构。



- 优点：适合大而深的复杂任务，如大型代码库重构、多系统迁移方案。
- 缺点：成本与延迟随层数指数上升，递归终止条件极难控制。
- 规则：**层数默认不超过 2 层**；每一层必须有预算和期限。

5. Blackboard (黑板模式)

结构：多个专业 Agent 共享一个结构化工作区（黑板），各自读取相关部分、写入结论，由控制组件或共识规则收敛结果。



- 优点：解耦彻底，适合需要多学科知识融合的开放问题。
- 缺点：黑板内容管理难，冲突消解复杂，最容易被忽视。
- 适合：诊断、研究综述、方案评审。
- 关键设计：写入权限、事实/假设分区、冲突解决规则、收敛截止时间。

6. Peer-to-Peer (对等协作)

结构：没有中心调度者，Agent之间平等对话、互相审查或协商。

- 优点：能产生对抗式质量提升，适合评审、辩论、模拟谈判。
- 缺点：通信开销最大，最难观测与终止。
- 使用建议：只作为局部审查结构，不让它成为全局拓扑。

三、模式对比表

维度	Orchestrator-Workers	Supervisor	Swarm	Hierarchical	Blackboard	P2P
控制方式	中心拆解	中心路由	链式交接	树形递归	共享空间	无中心
并行度	高	中	低	中高	高	中
动态性	低	中	高	中	高	最高
调试难度	低	中	中高	高	高	最高
Token 开销	中	中高	高	很高	高	最高
最适合	可并行的信息汇总	多角色动态诊断	客服转接	大型分层任务	多学科融合	评审辩论

四、通信设计：多 Agent 真正的成本在这里

1. 不要用自然语言“聊天”

Agent 之间自由聊天会产生三个问题：信息丢失、格式漂移、Token 爆炸。工程上应把 Agent 间通信当作**微服务接口**来设计。

2. 三种通信载体

载体	适用场景	优点	缺点
共享 State	同一进程 / 同一 Graph	零序列化成本，调试直接	耦合强，边界不清
结构化消息	跨进程 / 跨 Agent	可校验、可版本化、可审计	需要定义 Schema
自然语言摘要	需要人读的中间产物	可解释性好	不稳定，不宜作为系统契约

3. 一个最小交接 Payload

```
1 {
2   "task_id": "T-20260816-001",
3   "from": "triage-agent",
4   "to": "billing-agent",
5   "intent": "refund_request",
6   "context": {
7     "order_id": "O-88231",
8     "user_language": "zh-CN",
9     "attempts": 1
10  },
11  "acceptance_criteria": [
12    "确认订单状态",
13    "判断是否满足 7 天无理由退款",
14    "输出结构化处理建议"
15  ],
16  "max_tokens": 4000,
17  "deadline_ms": 60000
18 }
```

4. A2A 与 MCP 的分工

- **MCP**：解决 Agent 与外部工具/资源之间的接入问题；
- **A2A**：解决 Agent 与 Agent 之间的任务与状态交换问题；
- 两者不冲突：Worker 可以一边通过 MCP 使用工具，一边通过 A2A 契约接收任务和回传结果。

5. 通信设计检查单

- 每条消息是否有 Schema 和版本？
- 是否限制上下文随交接无限膨胀？
- 是否记录消息轨迹并支持回放？

- 是否有超时、重试、幂等和死信处理？
- 是否禁止 Agent 之间直接共享密钥或不可审计的隐式状态？

五、状态与上下文边界

多 Agent 系统最容易犯的错误是“全局共享一个上下文”。推荐三条边界：

1. **全局共享最小事实**：任务 ID、目标、最终交付物位置；
2. **局部上下文各管各的**：每个 Agent 只拿自己的 System Prompt、工具描述和必要任务卡；
3. **交接只传结果与验收标准**，不传完整推理过程，除非排查问题。

六、失败模式库

失败模式	表现	防御
目标漂移	子 Agent 越做越偏离原始任务	任务卡写清验收标准；Orchestrator 做终检
无限交接	客服场景 A→B→C→A 循环	最大交接次数、全局兜底策略
上下文稀释	消息越来越长，关键信息丢失	交接时只传结构化摘要
重复劳动	多个 Agent 同时做同一子任务	任务卡带排他与幂等 ID
成本爆炸	并行度失控或递归过深	全局 Token 预算、层数与并发上限
权限聚合	每个 Agent 权限都不大，组合后过大	全局最小权限，跨 Agent 动作单独审批
单点失效	Orchestrator 输出格式坏掉，全员空转	Schema 校验、重试、降级为固定 Workflow
结果互相矛盾	Blackboard/P2P 无法收敛	冲突消解规则、表决或仲裁 Agent、截止时间

七、最小实现示例：Orchestrator-Workers (伪代码)

```
1 function run_orchestrator(task):
2     plan = llm_plan(task, max_steps=5)           # 1. 生成任务卡列表
3     cards = validate_plan_schema(plan)           # 2. Schema 校验，失败重试一次
4     budget = Budget(max_total_tokens=50000)
5
6     results = parallel_map(cards, worker_call)    # 3. 并行分发给 Worker
7     report = llm_summarize(task, results)        # 4. 汇总结果
8     return validate_report(report, task)         # 5. 对照验收标准终检
9
10 function worker_call(card):
11     ctx = build_context(
12         system=card.role_prompt,                 # 只注入角色自己的提示词
13         task=card.objective,
14         acceptance=card.acceptance_criteria,
15         tools=card.allowed_tools                 # 只暴露白名单工具
16     )
17     for round in 1..card.max_rounds:
18         action = llm_decide(ctx)                 # 下一步：调工具或产出结果
19         if action.is_final:
20             return schema_validate(action.result, card.result_schema)
21         observation = safe_execute(action.tool_call)
```

```
22         ctx.append(observation)                                # 只追加观察，不追加
23         ↪ 闲聊
           raise WorkerTimeout(card)
```

这个骨架说明了多智能体系统与单 Agent 的关键差异：**计划、执行、校验被强制拆开，且每个 Worker 只能看到任务卡允许它看到的东西。**

八、选型决策表

场景特征	推荐模式	不推荐
多个独立子任务可并行	Orchestrator-Workers	P2P
客服多技能转接	Swarm	Hierarchical
复杂诊断、角色动态切换	Supervisor	Orchestrator-Workers
超大型分层任务	Hierarchical (2 层)	Swarm
多学科融合、结果难预先定义	Blackboard	Orchestrator-Workers
评审、辩论、对抗式质检	P2P 局部使用	全局 P2P
流程可提前确定	不要多 Agent，用 Workflow	任何多 Agent 模式

九、落地十步清单

1. 用单 Agent + Workflow 跑通最小闭环；
2. 从真实失败模式中找到必须拆分的边界；
3. 选择一种中心化模式起步，默认 Orchestrator-Workers；
4. 为任务卡和返回结果定义 Schema；
5. 给每个 Agent 独立 System Prompt、工具白名单与上下文边界；
6. 设置全局并发、层数、轮次、Token、超时预算；
7. 记录消息轨迹，支持按 task_id 回放；
8. 先离线用评测集验证，再做影子流量；
9. 准备好降级路径：多 Agent 失败回到单 Agent 或固定 Workflow；
10. 每次新增 Agent 都要重新评估成本、权限和调试复杂度。

十、延伸阅读

- [AI Agent 核心概念](#)：Multi-Agent、A2A 基础概念
- [AI workflow 中的 Workflow、Graph 与 Loop](#)：什么时候用 Workflow
- [一文搞懂 Harness Engineering](#)：Anthropic 三智能体架构案例
- [Agent 术语表与概念边界](#)：统一术语口径

一句话总结：**多智能体的价值来自边界清晰的分工，而不是 Agent 数量的堆叠。**